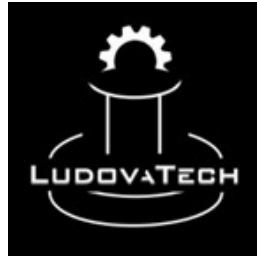


LUDOVATECH ROBOTICS TEAM



DESIGN DOCUMENT

Written by : Remy Magnon

June 13, 2024

Contents

1	INTRODUCTION	1
2	Team Overview and Funding	1
2.1	About our Team	1
2.2	Funding	1
3	HARDWARE	2
3.1	Original Design	2
3.2	Powertrain	3
3.3	Electronic Design	4
3.4	Mechanical Design	6
3.5	Dribbler and Kicker	6
4	SOFTWARE	7
4.1	Initial Code Foundations	7
4.2	LiDAR Detection System	8
4.3	Robot Positioning Within the Field	10
5	COMPETITIONS	11
5.1	Pre-Regional – The Surge	11
5.2	Regional Competition – Quest for Reliability	11
5.3	Nationals and School Presentation	13
6	Conclusion	14

List of Figures

3.1	Original 3D model of the robot.	2
3.2	BL2430 Brushless motor.	3
3.3	Motor assembly.	4
3.4	The original PCB design.	5
3.5	The new and improved PCB at nationals.	5
3.6	Von Mises stress and exaggerated displacement.	6
4.1	Ball detection using AI.	8
4.2	LiDAR LD19.	9
4.3	The results of the algorithm.	10
5.1	Regional competition.	12
5.2	National competition.	13

1 INTRODUCTION

This document provides a detailed presentation of the LudovaTech robotics team. More specifically, it provides a description of the team's project that made it to the RoboCup Junior 2024 world competition.

Over the course of this project, we explored different hardware and software solutions and built a collaborative team environment. From our initial designs to competition-day strategies, this document captures the technical intricacies and the learning moments of turning a vision into reality.

The remainder of this document is organized as follows: (a) Presentation of team organization and funding sponsors; (b) Hardware description and evolution: motors, electronics, structures and miscellaneous features; (c) Software description and evolution: from initial foundations to a mature code, ball detection system and field localization system; (d) Competitions: encountered challenges, implemented solutions, and final recognition earned.

2 Team Overview and Funding

2.1 About our Team

LudovaTech was founded in 2023. It all began with a robotics project initiated by the club's founder, when he was in middle school. He then decided to pursue the project in high school with a new team.

The LudovaTech robotics team comprises a total of five members for the 2023-2024 school year:

- Rémy Magnon, president & hardware specialist.
- Thomas Diot, software specialist.
- Yveline Liu, manufacturing & assembly specialist.
- Antoine Guilmot, software specialist

All team members gather every Monday to brainstorm ideas and allocate tasks.

2.2 Funding

Our club came to life with the support of several key contributors, especially our sponsors. Robot Maker and The Feed Reader Bot have funded some of our robot components. We are also grateful to MDL (Maison des Lycéens), a student organization at our high school, for providing funding for the national competition.

3 HARDWARE

3.1 Original Design

At the start of the project, we began with brainstorming sessions about what the robot should look like. We used the extremely useful RoboCup documentation [1] from past teams and came up with a few ideas regarding what should constitute the main components of the robot. Below is the list:

- A double ball-capturing zone (not shown in Figure 2.1 as it was made slightly later, when we changed to a single ball-capturing zone) to be able to catch the ball at the front and at the back of the robot.
- Two redundant batteries wired in parallel for greater capacity.
- A bottom PCB ¹ with surface-mounted LEDs and photosensitive sensors for line detection.
- A rotating LiDAR ² for accurate positioning of the robot within the playing field.
- A 4-motor powertrain—as it proved to be more efficient than the 3-motor version—equipped with omnidirectional wheels.
- A main PCB with all major electronic components like a Teensy 4.1 [2] as the main computing unit of the robot.
- An OpenMV H7 [3] camera pointed at a mirror protected inside an acrylic cylinder for 360° vision of the field.

Fig. 3.1 is a high-level depiction of the original robot.

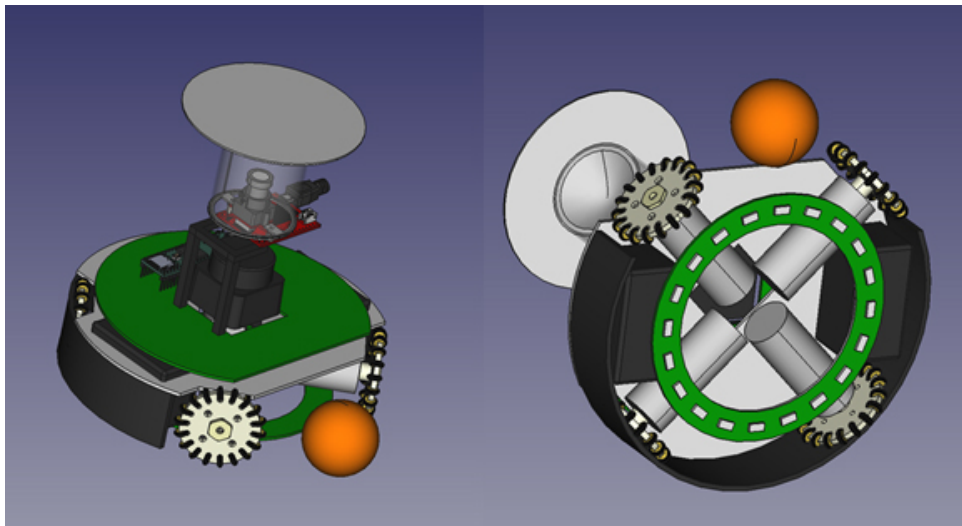


Figure 3.1: Original 3D model of the robot.

¹Printed Circuit Board.

²LiDAR (Light Detection and Ranging) is a remote sensing method that employs laser pulses to measure distances and generate precise representations of environments.

3.2 Powertrain

In November 2023, we started selecting the motors for the robot. We originally wanted to use Maxon EC-flat motors [4] to save space inside the robot and be a lot more efficient than the usual brushed-DC motors. But we quickly figured out that we could not afford the cost, more than 1000€ for a single powertrain. We would have had to make our own metal gears as well. But we stuck with brushless motors as it would teach us how to control them, and with the new rule change about voltage we wanted to try 24V motors. We ended up using the 6-watt BL2430 motor [5], GP24 version with a 16:1 planetary gearbox to reduce motor RPM from 6000 to 375, as shown in Fig. 3.2. These motors also have their own integrated drivers.

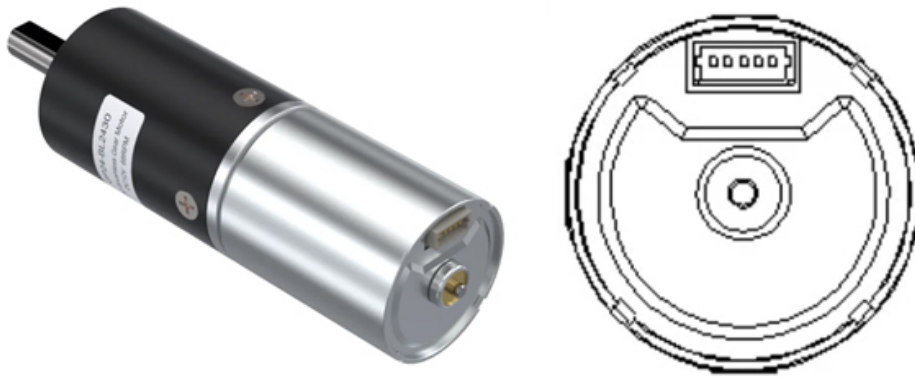


Figure 3.2: BL2430 Brushless motor.

The order (from left to right) and function of the pins is as follows:

- Ground Pin.
- Power Supply (24V).
- FG (Frequency Generator). This pin is directly connected to the hall sensors of the motor, and can be used to determine the speed of the motor. It sends a 50% duty cycle square wave signal whose frequency corresponds to the RPM of the motor (proportionally to the gearbox reduction ratio). We only added FG signal processing about a month prior to the international competition as it was not a priority early on in the project.
- PWM speed control (0-5V).
- CW/CCW (Clockwise/Counter clockwise, 0-5V). When the voltage is HIGH on this digital pin, the motor rotates clockwise, and when it is LOW, it rotates counter-clockwise.

Then, we chose 50mm wide omniwheels from GTFRobots [6]. They are mounted on the motor shaft with 3D printed PLA adaptors. We also had to make our own 3D printed

supports for the motors with extra reinforcements on the sides to make sure they don't bend under the weight of the motor. More about this topic in 2.4 Mechanical Design.

Originally, our motors were placed 90° from each other. But because we needed more space at the back for the battery and at the front for the kicker, we shifted them more toward the sides. They are now in a $70\text{--}110^\circ$ configuration, as shown in Fig. 3.3, which also increases the forward acceleration of the robot—a good thing to give more momentum to the ball prior to kicking.

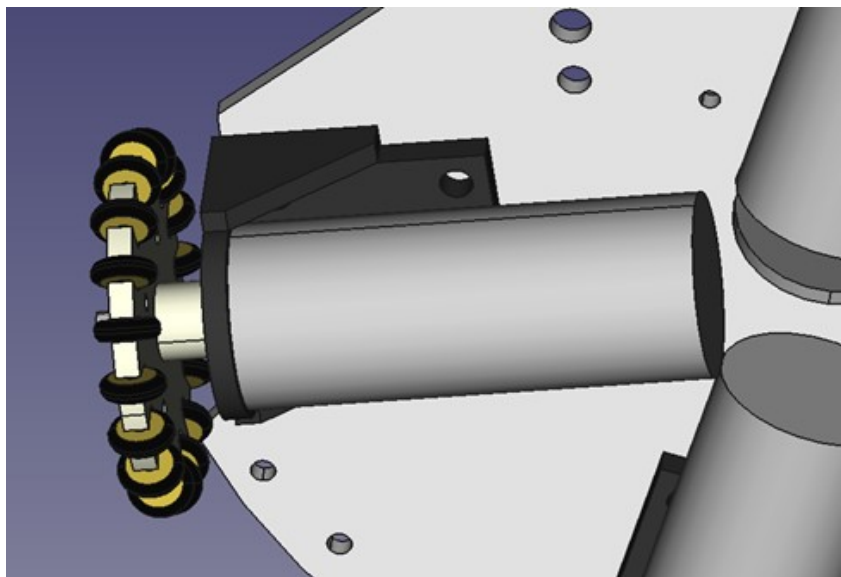


Figure 3.3: Motor assembly.

3.3 Electronic Design

Our original electronic design, Fig. 3.4, included a single PCB to power and connect all electronic components. We designed it on EasyEDA [7] thanks to the great LCSC component library and ordered them on JLCPCB. However, we had issues before and during the regional competition, as 24V current would leak into the 5V side of the circuit. That burnt a few Teensies and an OpenMV camera. The most likely cause of this issue is that we poorly soldered the JST-SH pins for the motors on the PCB, resulting in short circuits between the pins. We also probably had routing issues with the 5V regulator we used, which caused overvolting. Put simply, it was an electrical mess. For the national competition, we used a temporary solution to power the 5V side—as we had other issues to deal with—a 4.8V NiMH battery. We still included hardware for a regulator that we would eventually add back to the circuit later on.

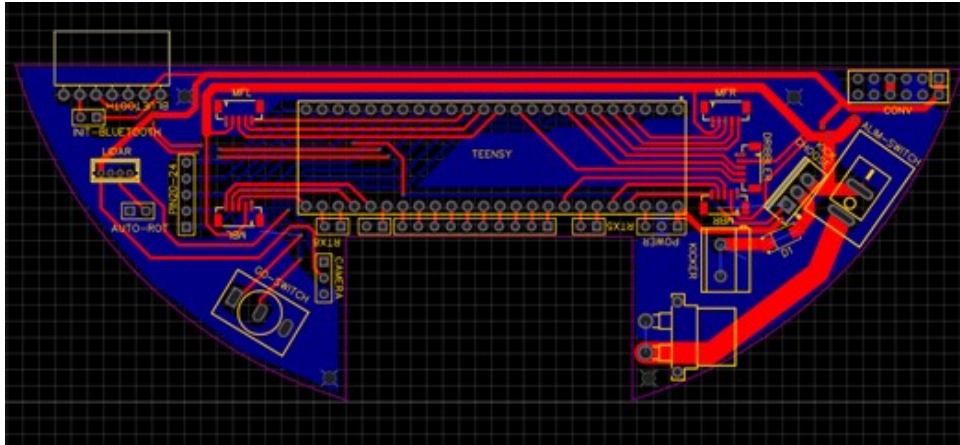


Figure 3.4: The original PCB design.

The day after the competition, we modeled new PCBs, splitting the 24V and 5V circuits into two separate boards. The 5V board is shown in Fig. 3.5. This method worked since we did not lose any components during the national competition. The 24V power board includes an XT-30 port for the battery, and JST-PH ports for the motors, because the pins are separated with a greater distance. This meant that we had to make our own cables to connect the JST-SH pins of the motors to the board (and the JST-XH of the LiDAR as well), which we created using a crimping tool.

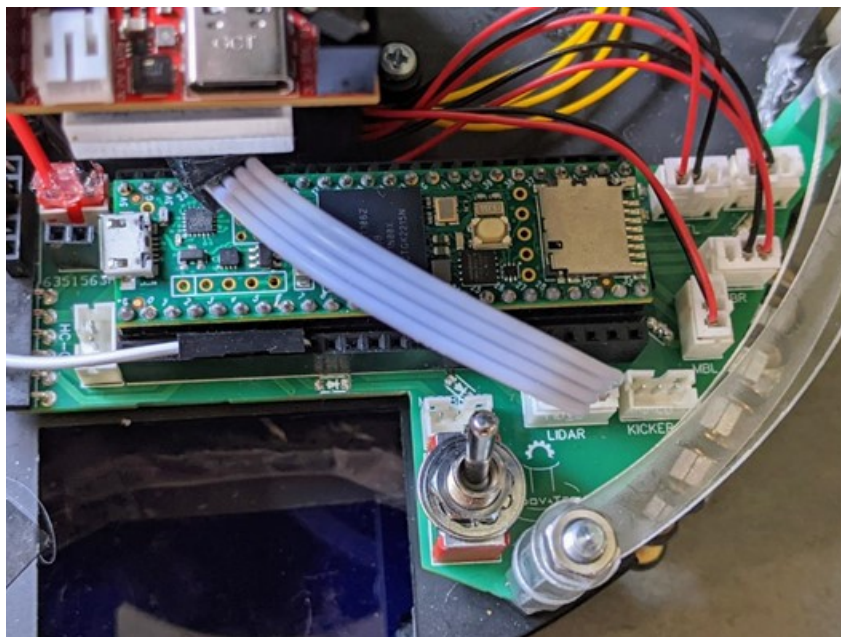


Figure 3.5: The new and improved PCB at nationals.

Originally, we wanted two 6-cell lipo batteries wired in parallel. We chose a battery with a capacity of 850mAh, and later switched to a single battery to reduce mass and complexity, while still having enough capacity for 2 consecutive matches with some margin. As we were struggling to correctly place the battery horizontally inside the robot, we decided we would place the battery vertically to save space and have better access to the cables.

3.4 Mechanical Design

To 3D model components of the robot, we decided to use FreeCAD [8] as it was a good first step to learn CAD software. We even modeled the field at the beginning of the year as we thought we would do simulations in Unity to choose the best strategies to use, but we unfortunately did not have enough time to do that.

We also used FreeCAD to perform structural simulations using the Finite Element Method [9] included in the software, as shown in Fig. 3.6. This feature helped us design structural reinforcements for the motors' supports, as they were bending under the weight of the robot being applied to the wheels. Even though it was probably not necessary (as we over-engineered the reinforcements) it helped us learn more about these simulation techniques.

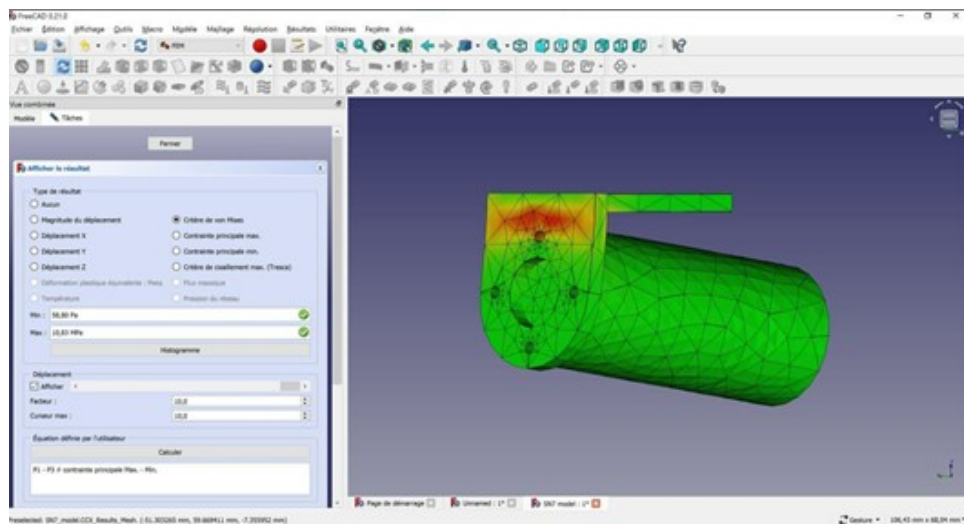


Figure 3.6: Von Mises stress and exaggerated displacement.

To manufacture parts with complex geometry, we used 3D printers from our school as well as a laser cutter for simpler parts such as the multiple layers of the chassis.

3.5 Dribbler and Kicker

For the dribbler, we decided early on that we would have the same motor we use for the wheels for extra redundancy. We simply selected ones with a lower reduction ratio as significantly less torque is required to maintain the ball than to move the robot around. Due to the distance between the motor and the dribbler roll, we didn't want to use gears (they also would have been hard to manufacture) so we rather used a pulley and belt system for mechanical transmission. The pulleys are both 16 mm diameter on the motor shaft and the dribbler roll, and they are connected with a 109 mm long belt.

The kicker remained low priority on our list for a long time, as it is not one of the most critical components (a robot without a kicker still works!). So it is not really optimized for kicking power. We still managed to get a decent kick, even more so with the momentum

of the robot. We use a relay to switch the kicker on or off, and it draws 4 A from the battery during 40ms. We also used a freewheel schottky diode to prevent the back-EMF of the inductor from damaging any component when the system is turned off.

4 SOFTWARE

The design of the robot took place over a long period, starting at the beginning of the year. Our code evolved in the same way, more or less following hardware progress.

From the start, our code has been divided into two parts. On one side, we have the camera code, written in MicroPython, which handles the detection of the ball and goals. On the other side, a C++ code runs on a Teensy 4.1, managing the rest: interpreting and processing data received from sensors, choosing the optimal strategy, and sending appropriate signals to the motors.

Both codes were entirely written by three programmers in the team: Thomas Diot, Remy Magnon and Antoine Guilmot.

4.1 Initial Code Foundations

At the beginning of the project, we explored two object detection methods for the camera.

Firstly, we tried to implement ball detection using machine learning with Edge Impulse [10]—software used for AI model integration into devices. We achieved very satisfactory results, Fig. 4.1, especially in changing lighting conditions. However, it came with two major limitations: the lack of scalability—as every modification required retraining the model—and low execution speed, as the AI model provided two to three times fewer position updates compared to the second method.

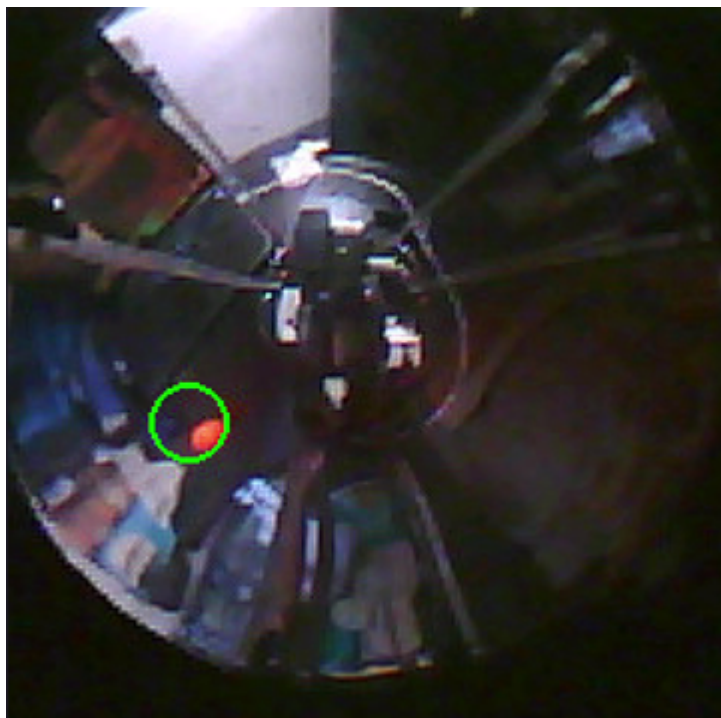


Figure 4.1: Ball detection using AI.

The second method, which we ended up selecting, is much more conventional. It involves using pixel color to identify the ball with an algorithm. We managed to mitigate the main disadvantage of this method—its dependency on ambient lighting—later in the code development by creating a real-time adaptation system for changing lighting conditions. Apart from this improvement, the ball and goal detection technique has remained more or less the same until today.

We started the Teensy C++ code by writing the basic classes we knew we would need—primarily a `Vector2` class for storing coordinates. A month later, we added the code to control the motors (see part 2.2), which remains unchanged to this day.

From the very beginning of the project, we have been using the Git version control system and storing our code in a GitHub organization. We can roughly measure our productivity by the number of commits created. Between September and the end of December, we had written nine commits.

4.2 LiDAR Detection System

Besides the fact that we started to see the regional competition approaching, with only three months left, we received the LiDAR [11], Fig. 4.2, and began trying to get it to work.



Figure 4.2: LiDAR LD19.

At this point, we were greatly disappointed. There were no tutorials on how to use these LD19 LiDARs. Worse still, the only existing documentation was an automatic English translation from a Chinese-written site.

Ironically, a precompiled—thus unreadable—software that allows us to visualize the points on a computer was provided. Therefore, we knew that the LiDAR worked and had a definition and precision that exceeded our expectations, but we had no way of understanding the encoded data it returned. Thomas kept working on the LiDAR until it was operational.

After a full week of work, using a lot of intuition and techniques approaching reverse engineering, he managed to obtain readable data that seemed correct, as the time variable incremented properly.

We then coded a small script to visualize the received data, and voilà. the data was correct. We added structures in the code to easily access these values. Thomas was happy, but something still bothered him—the LiDAR was sending a CRC-check mechanism in its data, but he hadn't yet managed to get it to work. He spent many more hours on this issue and finally found the solution: a line was missing in the precompiled value table in the documentation that allowed for verification! He ultimately found it in an obscure PDF in Chinese.

We then had a robust system for retrieving the LiDAR values, a code that has not been modified to this day.

Given the remarkable precision of the LiDAR, we took a gamble: we were convinced that

we could use this data to obtain the robot's position. We decided that, unlike most teams, we would not design a line detection system and would rely entirely on the LiDAR. This saved us a lot of time and energy, as it only requires a single sensor.

Though we had no idea at the time, this was a winning bet because at the nationals, when our system was ready, the robots almost never left the field. When they did, it was because the LiDAR cable was not properly connected!

4.3 Robot Positioning Within the Field

The final piece of code was added the day before the regional competition: LiDAR data analysis. We managed to code this very mathematical part of the code with the help of some classmates.

Using the Hough transform algorithm, we could detect lines among the point cloud. By ranking lines according to their accumulator size (how many points each one is made out of), we can find a wall by making the assumption that the line with the largest accumulator size is one of them (assumption that is true most of the time). We then check the angle and distance of other lines compared to that first wall we find, to determine a parallel wall and two perpendicular walls. The Hough transform gives us the polar coordinates of the lines, which we can easily convert to Cartesian coordinates. By finding the intersections of these lines, we can determine the coordinates of the corners of the field relative to the robot. From here on we can determine the robot's precise position on the field. The final results of the algorithm are represented in Fig. 4.3³.

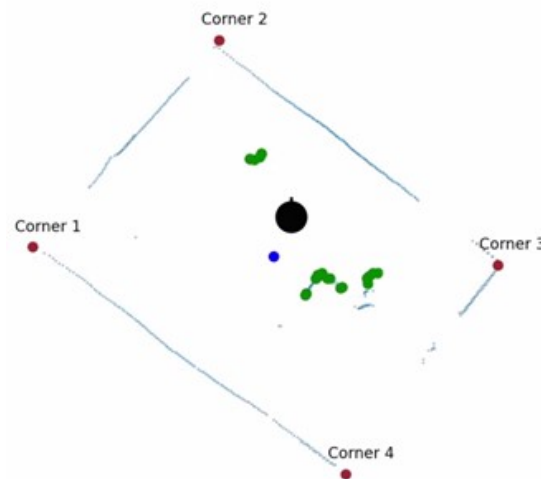


Figure 4.3: The results of the algorithm.

This method is extremely robust. When the algorithm finds a position for the robot, it is always mostly correct. When it does not detect one—which happens pretty rarely—we

³The blue lines represent the field's walls as detected by the LiDAR, the four corners result from the Hough Transform algorithm, and the green dots are other robots on the field.

can still determine the distance and angle from the nearest wall (the distance given in the Hough transform), thus still avoiding leaving the field.

5 COMPETITIONS

5.1 Pre-Regional – The Surge

Two weeks away from our regional competition in Paris, we had a working ball detection system, a LiDAR data processing system, and that was pretty much it (on the software side at least). Worse, the code for processing camera data could not run simultaneously with the LiDAR data processing code.

We then decided to take a potentially risky action we had been preparing for a long time. Until then, the C++ code was contained in a single .ino file; we decided to split the code into different .cpp and .hpp files, allowing us to sequentially execute the data processing from the camera and the LiDAR. We also switched to Visual Studio Code from the Arduino IDE we had been stuck with. Incredibly, the new code structure worked on the first try.

We added code to calculate the power required for each motor—based on the motor’s angle with the front of the robot and the direction of movement—to move the robot to a precise position on the field.

In addition to the new code, it is essential to remember that already written code often needs to be modified, either to be more robust or to integrate new functionalities.

Next, we added game strategies. Their operation had been discussed long in advance, but we had a new idea that saved a lot of time in coding and debugging. While it might seem logical to create very complex multi-step strategies for the robot, we realized that the strategies we had discussed could be integrated much more simply.

What makes testing normal strategies difficult, besides having to do it physically in the field, is that each strategy has several steps, requiring testing the robot’s interruptions at each step. So, we removed any memory the robot had of what it did previously. The robot determines the best place to go solely based on the information it receives from the sensors, without any memory of previous strategies. This means it will react normally even if the ball suddenly changes trajectory.

During this very short period, we wrote nearly a hundred commits, indicating the difference in pace compared to the first months.

5.2 Regional Competition – Quest for Reliability

The previous night, we miraculously finished the code, and the robot worked as expected. The next morning, we arrived at the competition, conducted a few tests, and everything

functioned perfectly. We then moved to the first match, as depicted in Fig. 5.1, and the robots stopped working! Always remember Murphy’s law when designing a robot or frankly anything that could fail.

Throughout the competition, we did our best to limit the damage. Various components ceased functioning one after another. We finished the last match with the most minimal code we could manage to have.



Figure 5.1: Regional competition.

We stayed until the closing ceremony and started making a list of everything needed to make the robot reliable.

To this day, we are unsure what exactly caused the robot to fail. Our most probable hypothesis is an electronic failure causing the Teensy to not run properly combined with bad software which could not manage the situation. But we could not determine a more precise cause and therefore decided to redesign most of the robot to maximize reliability.

At that time, as we worked on making each component and the entire code reliable, we did not even hope to qualify for the nationals. We were working for next year and a possible presentation in our school hall.

A few days later, as we had made good progress, we received the news: we were the last team qualified for the nationals!

Amid numerous modifications to make the code reliable, such as adding Radians and Degrees classes for automatic conversions and a logging system, one change was key. We decided to implement a system of unit tests to ensure the robot’s code reliability. However, we wanted the tests to run on the computer and not on the board. Through a particular CMake configuration, we managed to make the code believe that `Arduino.h` was one of our files. Thus, we emulated `Arduino.h` to enable our tests to run successfully.

We also made various attempts to rewrite the LiDAR data analysis system, but ultimately we kept the original code since it worked well, even though probably not fully optimal.

At the same time, given the growing number of tasks, we decided to fully utilize GitHub's features, using issues and projects, an agile system to manage task importance and assignment.

We reached 230 commits, demonstrating the extent of the modifications made.

5.3 Nationals and School Presentation

We managed to find the last bugs the night before the nationals, but this time they were small bugs and not issues affecting the entire code functionality.

We then participated in the national competition, Fig. 5.2. Apart from some inconveniences due to changing light conditions (which cost us some goals), we did not detect any major bugs. We won all our matches with a good lead and conceded only a few goals throughout the competition. The team finished as champion of France and qualified for the international competition!



Figure 5.2: National competition.

Similarly, during the school presentation, everything went as planned except for a few problems caused by excessive battery drain because the robots operated for a long time without a break.

Since nationals, we have been looking forward to add new features to the robot while maintaining its reliability. We reviewed the strategy selection system to make it more robust and developed a utility to visualize the robot's actions from its logs.

The new features primarily involve adding Bluetooth communication between the robots, a dribbler, and a kicker. We also aim to further improve the strategies, and are still working on these features as we are writing these lines.

Thanks to a durable field we built for the school presentation, we are also able to conduct more tests.

6 Conclusion

From overcoming technical challenges to achieving national recognition, the LudovaTech team has evolved significantly over the past couple of years. Our journey from the initial design stages to competing at the national level has been marked by continuous learning, innovation, and resilience. Despite the numerous obstacles we faced, from hardware malfunctions to software bugs, each challenge provided us with valuable lessons that ultimately strengthened our team and improved our robots.

Our success in the national competition, where we emerged as France's champions, is a testament to our dedication and teamwork. We are now preparing for the RoboCup Junior 2024 international competition, we are working hard to improve our robots through implementing new features, refining our game strategies, and further enhancing the overall reliability of the robot.

We hope this document will serve as a valuable resource for future robotics team.

References

- [1] RoboCup Junior Soccer Resources. Available at: <https://github.com/robocup-junior/awesome-rcj-soccer>.
- [2] PJRC Teensy 4.1 Microcontroller Documentation. Available at: <https://www.pjrc.com/store/teensy41.html>.
- [3] OpenMV H7 Camera Product Page. Available at: <https://openmv.io/collections/cameras/products/openmv-cam-h7-r2>.
- [4] Maxon EC-flat 45 Product Page. Available at: <https://www.maxongroup.com/maxon/view/product/200188>.
- [5] BL2430 GP24 Motor Product Page. Available at: <https://fr.aliexpress.com/i/1005005402356401.html>.
- [6] GTFRobots 50mm Omniwheel Product Page. Available at: <https://gtfroboots.com/product/50mm-diameter-8mm-hole-size-omni-wheel>.
- [7] EasyEDA - Online PCB Design Tool. Available at: <https://easyeda.com>.
- [8] FreeCAD - Computer Aided Design Tool. Available at: <https://www.freecad.org/index.php>.
- [9] FreeCAD Finite Element Model Workbench Documentation. Available at: https://wiki.freecad.org/FEM_Workbench.
- [10] Edge Impulse - Machine Learning Platform. Available at: <https://edgeimpulse.com>.
- [11] Rotating LiDAR LD19 Documentation. Available at: https://www.waveshare.com/wiki/DT0F_LiDAR_LD19.